시스템 프로그래밍

8장. 프로세스

컴퓨터소프트웨어학부

School of Computer Software at Daegu Catholic University © 2021

목차

- 실습 문제
  - ✓ 1. 명령인자(arguments) 입력 받고 출력하기
  - ✓ 2. 환경변수 출력하기
  - ✓ 3. 특정 환경변수 명시 후 출력하기
  - ✓ 4. exit 처리기
  - ✓ 5. 현재 프로세스 id와 부모프로세스 id 출력하기
  - ✓ 6. 프로세스의 사용자 ID와 그룹 ID 출력하기
  - ✓ 7. 프로세스 출력 명령어 사용하기
  - ✓ 8. 전면 처리, 후면처리 예시 따라하기
  - ✓ 9. wait 명령어로 프로세스 기다리기
- 과제
  - ✓ 1. 매개인자 입력 후 인자에 해당하는 값 출력하기
  - ✓ 2. 현재 프로세스와 부모프로세스의 관계 파악하기
  - ✓ 3. 예제 출력을 통해 fork() 시스템 호출의 수행역할 파악하기

실습 1 – 명령인자(arguments) 입력 받고 출력하기

- 명령인자 입출력 프로그램 구현

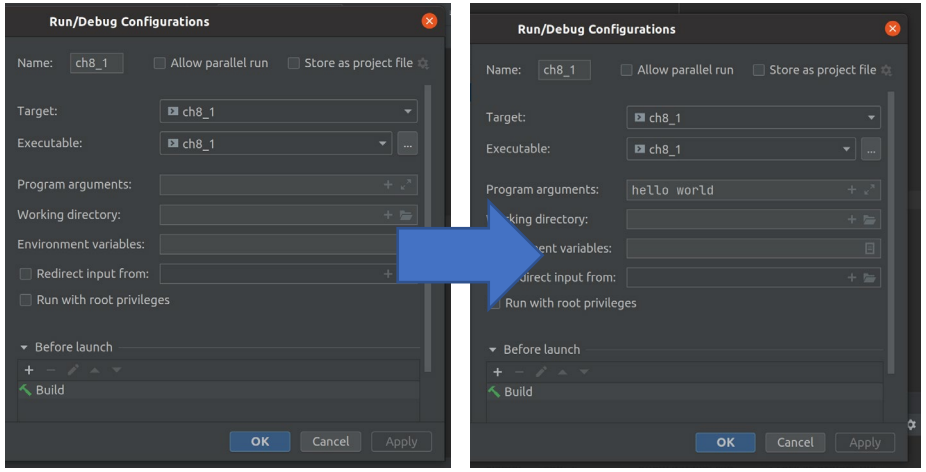
✓참조 자료 : 강의 자료 18번 슬라이드, 교재 252쪽

1. Clion IDE에서 Edit Configuration -> Run/Debug Configuration 의 Program arguments에 명령인자 입력 or Terminal에서 명령인자 입력

❖ 실행 명령어

- gcc –o main main.c
- ./main hello world

실습 1 – 명령인자(arguments) 입력 받고 출력하기



명령인자 입력

```
/home/raynor/CLionProjects/ch8-1/cmake-build-debug/ch8_1 hello world
argv[0]: /home/raynor/CLionProjects/ch8-1/cmake-build-debug/ch8_1
argv[1]: hello
argv[2]: world

Process finished with exit code 0
```

프로그램 실행결과

```
raynor@ubuntu:~/CLionProjects/ch8-1$ ls
cmake-build-debug  CMakeLists.txt  main.c
raynor@ubuntu:~/CLionProjects/ch8-1$ gcc -o main main.c
raynor@ubuntu:~/CLionProjects/ch8-1$ ls
cmake-build-debug  CMakeLists.txt  main  main.c
```

```
raynor@ubuntu:~/CLionProjects/ch8-1$ ./main hello world
argv[0]: ./main
argv[1]: hello
argv[2]: world
```

터미널 명령인자 입출력결과

실습 2 – 환경변수 출력하기

- 환경변수 출력 프로그램을 구현하세요.

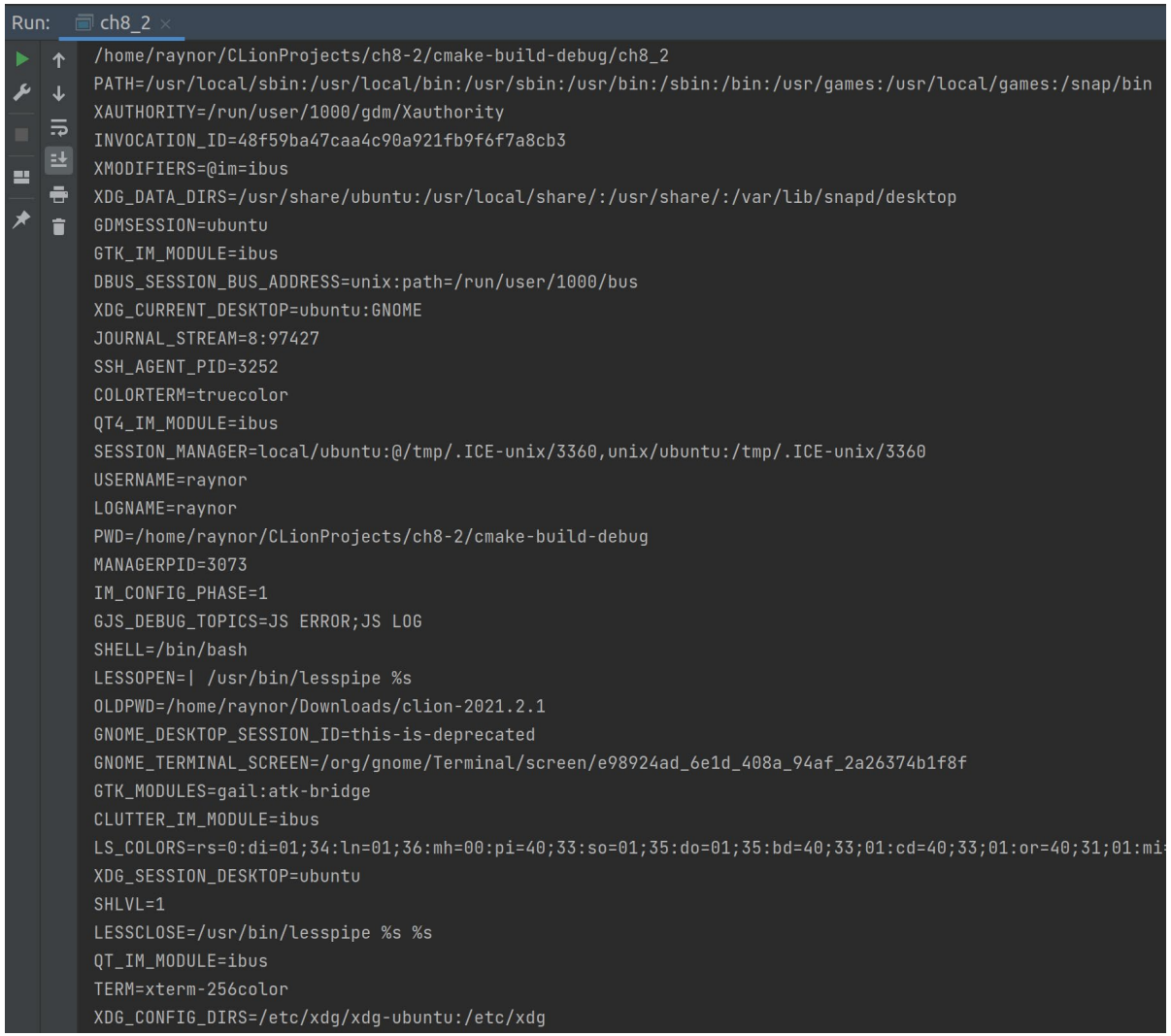
✓참고자료 : 강의 자료 19번 슬라이드, 교재 254쪽

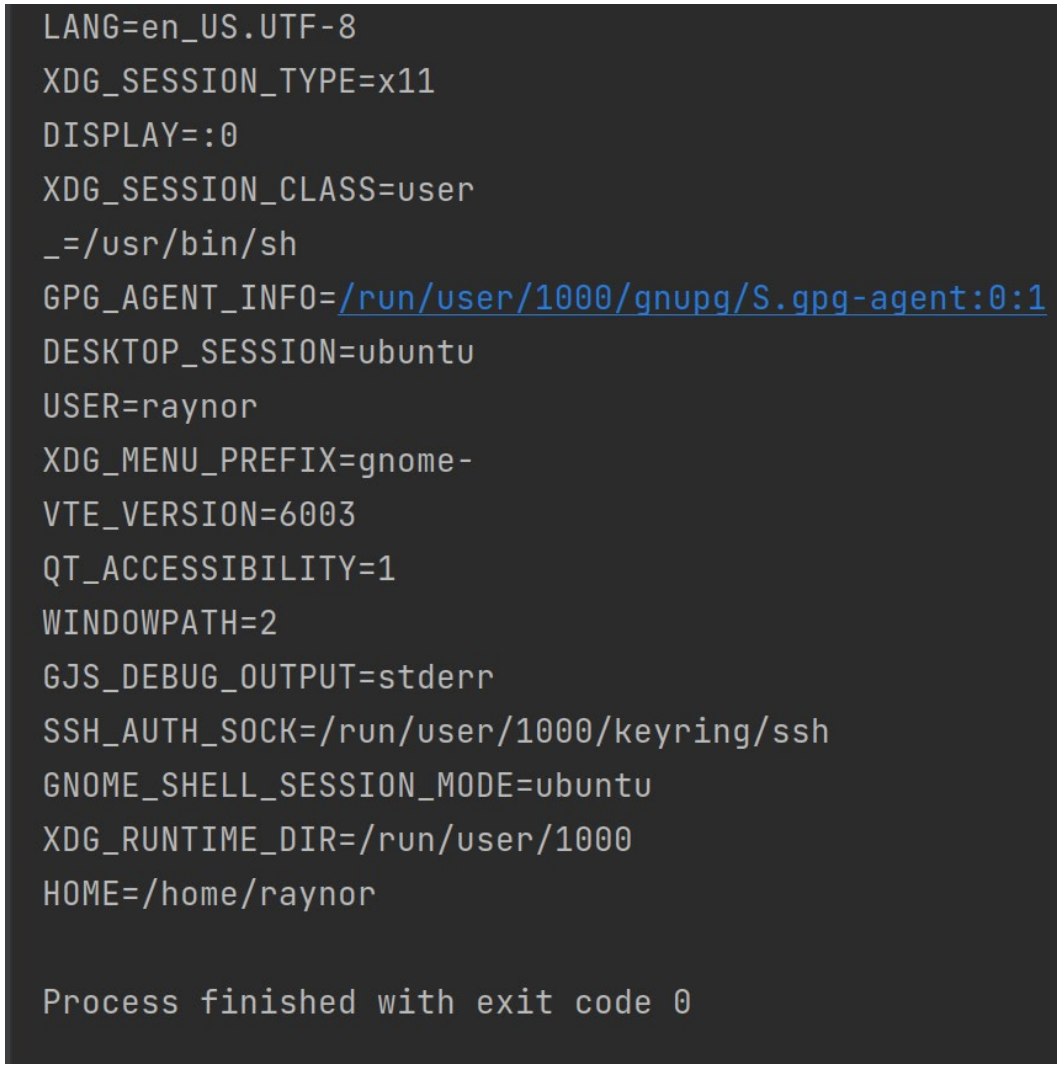
1. 전역변수 environ을 사용하여 환경변수 전체에 접근한다.

❖ 실행 명령어

- gcc –o main main.c
- ./main

실습 2 – 환경변수 출력하기





Clion IDE 에서 결과 출력

실습 2 – 환경변수 출력하기

```
raynor@ubuntu:~/CLionProjects/ch8-2$ ls
cmake-build-debug  CMakeLists.txt  main.c
raynor@ubuntu:~/CLionProjects/ch8-2$ gcc -o main main.c
raynor@ubuntu:~/CLionProjects/ch8-2$ ls
cmake-build-debug  CMakeLists.txt  main  main.c
raynor@ubuntu:~/CLionProjects/ch8-2$ ./main
```

```
SHELL=/bin/bash
SESSION_MANAGER=local/ubuntu:@/tmp/.ICE-unix/3360,unix/ubuntu:/tmp/.ICE-unix/3360
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GTK_IM_MODULE=ibus
QT4_IM_MODULE=ibus
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
SSH_AGENT_PID=3252
GTK_MODULES=gail:atk-bridge
PWD=/home/raynor/CLionProjects/ch8-2
LOGNAME=raynor
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=x11
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
XAUTHORITY=/run/user/1000/gdm/Xauthority
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
WINDOWPATH=2
HOME=/home/raynor
USERNAME=raynor
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:
;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*
:*.xz=01;31:*.zst=01;31:*.tzst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=
:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=
1;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:
ebm=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01
dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;
.ogg=00;36:*.ra=00;36:*.wav=00;36:*.oga=00;36:*.opus=00;36:*.spx=00;36:*.xspf=00;36:
XDG_CURRENT_DESKTOP=ubuntu:GNOME
VTE_VERSION=6003
GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/1a572464_e998_4de7_b271_d1a763e4d789
INVOCATION_ID=48f59ba47caa4c90a921fb9f6f7a8cb3
MANAGERPID=3073
CLUTTER_IM_MODULE=ibus
GJS_DEBUG_OUTPUT=stderr
LESSCLOSE=/usr/bin/lesspipe %s %s
XDG_SESSION_CLASS=user
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=raynor
GNOME_TERMINAL_SERVICE=:1.83
DISPLAY=:0
```

Terminal 에서 결과 출력

실습 3 - 특정 환경변수 명시 후 출력하기

- 특정환경변수를 지정하여 출력하는 프로그램을 구현하세요.

✓참고자료 : 강의 자료 21번 슬라이드, 교재 255쪽

1. 특정 환경 변수에 접근할 수 있는 getenv();를 이용하여 “HOME”, “SHELL”, “PATH” 환경변수를 출력한다.

❖ 실행 명령어

- gcc –o main main.c
- ./main

실습 3 - 특정 환경변수 명시 후 출력하기

```
raynor@ubuntu:~/CLionProjects/ch8-3$ ls
cmake-build-debug  CMakeLists.txt  main.c
raynor@ubuntu:~/CLionProjects/ch8-3$ gcc -o main main.c
raynor@ubuntu:~/CLionProjects/ch8-3$ ls
cmake-build-debug  CMakeLists.txt  main  main.c
raynor@ubuntu:~/CLionProjects/ch8-3$ ./main
HOME = /home/raynor
SHELL = /bin/bash
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

Terminal 에서 결과 출력

```
/home/raynor/CLionProjects/ch8-3/cmake-build-debug/ch8_3
HOME = /home/raynor
SHELL = /bin/bash
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin

Process finished with exit code 0
```

Clion IDE 에서 결과 출력

실습 4 – exit 처리기

- Exit 처리기(exit handler) 함수를 등록하는 프로그램을 구현하시오.

✓참고자료 : 강의 자료 23~28번 슬라이드, 교재 258~259쪽

1. Exit 처리기 함수를 static 함수로 선언

2. Atexit() 시스템 호출을 이용하여 Exit 처리기 함수 등록

❖ 실행 명령어

- gcc –o main main.c
- ./main

실습 4 – exit 처리기

```
raynor@ubuntu:~/CLionProjects/ch8_4$ ls
cmake-build-debug  CMakeLists.txt  main.c
raynor@ubuntu:~/CLionProjects/ch8_4$ gcc -o main main.c
raynor@ubuntu:~/CLionProjects/ch8_4$ ls
cmake-build-debug  CMakeLists.txt  main  main.c
raynor@ubuntu:~/CLionProjects/ch8_4$ ./main
main done
second exit handler
first exit handler
```

Terminal 에서 결과 출력

```
/home/raynor/CLionProjects/ch8_4/cmake-build-debug/ch8_4
main done
second exit handler
first exit handler

Process finished with exit code 0
```

Clion IDE 에서 결과 출력

실습 5 – 현재 프로세스 id와 부모프로세스 id 출력하기

- Getpid(), getppid()를 이용하여 현재 프로세스 id와 부모프로세스 id 출력프로그램을 구현하시오

  ✓참고자료 : 강의 자료 31번 슬라이드, 교재 261쪽

  1. Getpid()를 이용하여 현재 프로세스 id를 출력
  2. Getppid()를 이용하여 부모프로세스 id를 출력

  ❖ 실행 명령어

  - gcc –o main main.c
  - ./main

실습 5 – 현재 프로세스 id와 부모프로세스 id 출력하기

```
raynor@ubuntu:~/CLionProjects/ch8-5$ ls
cmake-build-debug  CMakeLists.txt  main.c
raynor@ubuntu:~/CLionProjects/ch8-5$ gcc -o main main.c
raynor@ubuntu:~/CLionProjects/ch8-5$ ls
cmake-build-debug  CMakeLists.txt  main  main.c
raynor@ubuntu:~/CLionProjects/ch8-5$ ./main
my process number: [6044]
my parent process number: [5598]
raynor@ubuntu:~/CLionProjects/ch8-5$
```

Terminal 에서 결과 출력

```
/home/raynor/CLionProjects/ch8-5/cmake-build-debug/ch8_5
my process number: [6027]
my parent process number: [4367]

Process finished with exit code 0
```

Clion IDE 에서 결과 출력

실습 6 – 프로세스의 사용자 ID와 그룹 ID 출력하기

- 프로세스 사용ID와 그룹 ID 출력 함수를 사용하시오.

✓참고자료 : 강의 자료 32-36번 슬라이드, 교재 263쪽

1. 프로세스는 프로세스 ID 외 프로세스를 실행시킨 사용자 ID와 사용자가 포함된 그룹 ID를 갖는다. 이는 프로세스가 수행할 수 있는 연산을 결정하는데 사용한다.
2. 사용자, 그룹 ID는 각각 실제와 유효 id로 구분할 수 있다.

➢ Getuid(), geteuid(), getgid(), getegid() 사용

❖ 실행 명령어

- gcc –o main main.c
- ./main

실습 6 – 프로세스의 사용자 ID와 그룹 ID 출력하기

```
raynor@ubuntu:~/CLionProjects/ch8-6$ ls
cmake-build-debug  CMakeLists.txt  main.c
raynor@ubuntu:~/CLionProjects/ch8-6$ gcc -o main main.c
raynor@ubuntu:~/CLionProjects/ch8-6$ ls
cmake-build-debug  CMakeLists.txt  main  main.c
raynor@ubuntu:~/CLionProjects/ch8-6$ ./main
My Realistic User ID: 1000(raynor)
My Valid User ID: 1000(raynor)
My Realistic Group ID: 1000(raynor)
My Valid Group ID: 1000(raynor)
```

Terminal 에서 결과 출력

```
/home/raynor/CLionProjects/ch8-6/cmake-build-debug/ch8_6
My Realistic User ID: 1000(raynor)
My Valid User ID: 1000(raynor)
My Realistic Group ID: 1000(raynor)
My Valid Group ID: 1000(raynor)

Process finished with exit code 0
```

Clion IDE 에서 결과 출력

실습 7 – 프로세스 출력 명령어 사용하기

- 현재 존재하는 프로세스들의 실행상태를 요약하여 출력하는 명령어 “ps”를 사용한다.

✓참고자료 : 강의 자료 8-9번 슬라이드, 교재 246-247쪽

1. Ps : 현재 존재하는 프로세스 실행상태를 요약 출력
2. Ps –f : 프로세스에 대하여 더 자세한 정보를 출력
3. Ps -aux
    1. -a : 모든 사용자의 프로세스 출력
    2. -u : 프로세스에 대한 더 자세한 정보 출력
    3. -x : 더 이상 제어 터미널을 갖지 않은 프로세스들도 함께 출력
4. Ps –ef
    1. -e : 모든 사용자 프로세스 정보를 출력
    2. -f : 프로세스에 대한 좀 더 자세한 정보 출력

실습 7 – 프로세스 출력 명령어 사용하기

```
raynor@ubuntu:~$ ps
    PID TTY          TIME CMD
   6366 pts/2    00:00:00 bash
   6372 pts/2    00:00:00 ps
```

ps

```
raynor@ubuntu:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
raynor      6366    4132  0 04:25 pts/2    00:00:00 bash
raynor      6373    6366  0 04:26 pts/2    00:00:00 ps -f
```

Ps -f

실습 7 – 프로세스 출력 명령어 사용하기

```
raynor@ubuntu:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 Oct18 ?        00:00:05 /sbin/init auto noprompt
root           2       0  0 Oct18 ?        00:00:00 [kthreadd]
root           3       2  0 Oct18 ?        00:00:00 [rcu_gp]
root           4       2  0 Oct18 ?        00:00:00 [rcu_par_gp]
root           6       2  0 Oct18 ?        00:00:00 [kworker/0:0H-events_highpri]
root           9       2  0 Oct18 ?        00:00:00 [mm_percpu_wq]
root          10       2  0 Oct18 ?        00:00:00 [rcu_tasks_rude_]
root          11       2  0 Oct18 ?        00:00:00 [rcu_tasks_trace]
root          12       2  0 Oct18 ?        00:00:04 [ksoftirqd/0]
root          13       2  0 Oct18 ?        00:00:02 [rcu_sched]
root          14       2  0 Oct18 ?        00:00:00 [migration/0]
root          15       2  0 Oct18 ?        00:00:00 [idle_inject/0]
root          16       2  0 Oct18 ?        00:00:00 [cpuhp/0]
root          17       2  0 Oct18 ?        00:00:00 [cpuhp/1]
root          18       2  0 Oct18 ?        00:00:00 [idle_inject/1]
root          19       2  0 Oct18 ?        00:00:01 [migration/1]
root          20       2  0 Oct18 ?        00:00:00 [ksoftirqd/1]
root          22       2  0 Oct18 ?        00:00:00 [kworker/1:0H-events_highpri]
root          23       2  0 Oct18 ?        00:00:00 [kdevtmpfs]
root          24       2  0 Oct18 ?        00:00:00 [netns]
root          25       2  0 Oct18 ?        00:00:00 [inet_frag_wq]
root          26       2  0 Oct18 ?        00:00:00 [kauditd]
root          28       2  0 Oct18 ?        00:00:00 [khungtaskd]
root          29       2  0 Oct18 ?        00:00:00 [oom_reaper]
root          30       2  0 Oct18 ?        00:00:00 [writeback]
root          31       2  0 Oct18 ?        00:00:02 [kcompactd0]
root          32       2  0 Oct18 ?        00:00:00 [ksmd]
root          33       2  0 Oct18 ?        00:00:00 [khugepaged]
root          80       2  0 Oct18 ?        00:00:00 [kintegrityd]
root          81       2  0 Oct18 ?        00:00:00 [kblockd]
```

Ps -ef

```
raynor@ubuntu:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 168900 11636 ?        Ss   Oct18   0:05 /sbin/init auto noprompt
root           2  0.0  0.0      0     0 ?        S    Oct18   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   Oct18   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   Oct18   0:00 [rcu_par_gp]
root           6  0.0  0.0      0     0 ?        I<   Oct18   0:00 [kworker/0:0H-events_highpri]
root           9  0.0  0.0      0     0 ?        I<   Oct18   0:00 [mm_percpu_wq]
root          10  0.0  0.0      0     0 ?        S    Oct18   0:00 [rcu_tasks_rude_]
root          11  0.0  0.0      0     0 ?        S    Oct18   0:00 [rcu_tasks_trace]
root          12  0.0  0.0      0     0 ?        S    Oct18   0:04 [ksoftirqd/0]
root          13  0.0  0.0      0     0 ?        I    Oct18   0:02 [rcu_sched]
root          14  0.0  0.0      0     0 ?        S    Oct18   0:00 [migration/0]
root          15  0.0  0.0      0     0 ?        S    Oct18   0:00 [idle_inject/0]
root          16  0.0  0.0      0     0 ?        S    Oct18   0:00 [cpuhp/0]
root          17  0.0  0.0      0     0 ?        S    Oct18   0:00 [cpuhp/1]
root          18  0.0  0.0      0     0 ?        S    Oct18   0:00 [idle_inject/1]
root          19  0.0  0.0      0     0 ?        S    Oct18   0:01 [migration/1]
root          20  0.0  0.0      0     0 ?        S    Oct18   0:00 [ksoftirqd/1]
root          22  0.0  0.0      0     0 ?        I<   Oct18   0:00 [kworker/1:0H-events_highpri]
root          23  0.0  0.0      0     0 ?        S    Oct18   0:00 [kdevtmpfs]
root          24  0.0  0.0      0     0 ?        I<   Oct18   0:00 [netns]
root          25  0.0  0.0      0     0 ?        I<   Oct18   0:00 [inet_frag_wq]
root          26  0.0  0.0      0     0 ?        S    Oct18   0:00 [kauditd]
root          28  0.0  0.0      0     0 ?        S    Oct18   0:00 [khungtaskd]
root          29  0.0  0.0      0     0 ?        S    Oct18   0:00 [oom_reaper]
root          30  0.0  0.0      0     0 ?        I<   Oct18   0:00 [writeback]
root          31  0.0  0.0      0     0 ?        S    Oct18   0:02 [kcompactd0]
root          32  0.0  0.0      0     0 ?        SN   Oct18   0:00 [ksmd]
root          33  0.0  0.0      0     0 ?        SN   Oct18   0:00 [khugepaged]
root          80  0.0  0.0      0     0 ?        I<   Oct18   0:00 [kintegrityd]
root          81  0.0  0.0      0     0 ?        I<   Oct18   0:00 [kblockd]
root          82  0.0  0.0      0     0 ?        I<   Oct18   0:00 [blkcg_punt_bio]
root          83  0.0  0.0      0     0 ?        I<   Oct18   0:00 [tpm_dev_wq]
root          84  0.0  0.0      0     0 ?        I<   Oct18   0:00 [ata_sff]
root          85  0.0  0.0      0     0 ?        I<   Oct18   0:00 [md]
root          86  0.0  0.0      0     0 ?        I<   Oct18   0:00 [edac-poller]
root          87  0.0  0.0      0     0 ?        I<   Oct18   0:00 [devfreq_wq]
root          88  0.0  0.0      0     0 ?        S    Oct18   0:00 [watchdogd]
root          91  0.0  0.0      0     0 ?        I<   Oct18   0:00 [kworker/1:1H-kblockd]
root          93  0.0  0.0      0     0 ?        S    Oct18   0:00 [kswapd0]
```

Ps -aux

실습 8 – 전면 처리, 후면처리 예시 따라하기

- 전면, 후면 처리 명령어를 사용하여 출력 결과를 확인 후 설명하시오.

✓참고자료 : 6-7 슬라이드 확인, 교재 243-245쪽

1. 명령줄 인수로 받은 파일의 버퍼 사용 방식을 조사하여 출력하세요.
2. stdin, stdout, stderr, 파일이름을 명령어 인수로 받을 수 있다.
3. stdin, stdout, stderr, 파일의 버퍼 사용 방식을 출력하시오.
4. 예외처리
   - 파일이 열리지 않으면 오류 출력

❖ 실행 명령어

- $(echoSTART1;sleep10;echoDONE1)&
- $fg%1
- $(echoSTART2;sleep10;echoDONE2)
- ^z
- $bg%1

실습 8 – 전면 처리, 후면처리 예시 따라하기

```
raynor@ubuntu:~$ (echo START1; sleep 10; echo DONE1) &
[1] 6497
START1
raynor@ubuntu:~$ fg %1
( echo START1; sleep 10; echo DONE1 )
DONE1
raynor@ubuntu:~$ (echo START2; sleep 10; echo DONE2)
START2
^Z
[1]+  Stopped                 ( echo START2; sleep 10; echo DONE2 )
raynor@ubuntu:~$ bg %1
[1]+ ( echo START2; sleep 10; echo DONE2 ) &
raynor@ubuntu:~$ DONE2
```

실습 9 – wait 명령어로 프로세스 기다리기

- 아래 명령어를 실행하고 출력 결과에 대해 설명하시오.

✓참고자료 : 12 슬라이드, 교재 249쪽 확인

1. Wait 명령어를 이용하여 특정 프로세스가 종료될 때까지 기다린다. 프로세스 번호를 지정하지 않으면 모든 자식 프로세스를 기다린다.

❖ 실행 명령어

- $(echoSTART1;sleep10;echoDONE1)&
- $(echoSTART2;sleep10;echoDONE2)&
- $echo1;wait;echo2

실습 9 – wait 명령어로 프로세스 기다리기

```
raynor@ubuntu:~/Downloads/clion-2021.2.1/bin$ (echo START1; sleep 10; echo DONE1) &
[1] 40421
START1
raynor@ubuntu:~/Downloads/clion-2021.2.1/bin$ DONE1
^C
[1]+  Done                    ( echo START1; sleep 10; echo DONE1 )
raynor@ubuntu:~/Downloads/clion-2021.2.1/bin$ (echo START2; sleep 10; echo DONE2) &
[1] 40425
raynor@ubuntu:~/Downloads/clion-2021.2.1/bin$ START2
DONE2
^C
[1]+  Done                    ( echo START2; sleep 10; echo DONE2 )
```

```
raynor@ubuntu:~/Downloads/clion-2021.2.1/bin$ echo 1; wait; echo 2;
1
2
```

과제 1 – 매개인자 입력 후 인자에 해당하는 값 출력하기

- 프로세스의 실행환경에 대한 정보를 제공하기 위한 프로그램을 구현한다. 다음 옵션에 따라 해당되는 값을 출력하는 프로그램을 작성하라.
- -e[환경변수이름] : 지정된 이름의 환경변수 값, 이름을 제외하는 경우 모든 환경변수들의 값을 출력한다.
- -u :실행 중인 프로세스의 실제 사용자 ID와 유효 사용자 ID
- -g : 실행 중인 프로세스의 실제 그룹 ID와 유효 그룹 ID
- - i : 프로세스 ID
- - p : 부모프로세스 ID
- *** 각 매개인자에 대해 개별 프로그램이 아닌 하나의 프로그램에서 6가지 경우를 모두 처리하는 프로그램을 구현한다.**

과제 1 – 매개인자 입력 후 인자에 해당하는 값 출력하기

매개인자 –e (환경변수이름제외)

```
/home/raynor/CLionProjects/ch8-10/cmake-build-debug/ch8_10 -e
-e EMPTY
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
XAUTHORITY=/run/user/1000/gdm/Xauthority
INVOCATION_ID=48f59ba47caa4c90a921fb9f6f7a8cb3
XMODIFIERS=@im=ibus
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/local/share/:/usr/share/:/var/lib/snapd/desktop
GDMSESSION=ubuntu
GTK_IM_MODULE=ibus
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
XDG_CURRENT_DESKTOP=ubuntu:GNOME
JOURNAL_STREAM=8:97427
SSH_AGENT_PID=3252
COLORTERM=truecolor
QT4_IM_MODULE=ibus
SESSION_MANAGER=local/ubuntu:@/tmp/.ICE-unix/3360,unix/ubuntu:/tmp/.ICE-unix/3360
USERNAME=raynor
LOGNAME=raynor
PWD=/home/raynor/CLionProjects/ch8-10/cmake-build-debug
MANAGERPID=3073
IM_CONFIG_PHASE=1
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
SHELL=/bin/bash
LESSOPEN=| /usr/bin/lesspipe %s
OLDPWD=/home/raynor/Downloads/clion-2021.2.1
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/e98924ad_6e1d_408a_94af_2a26374b1f8f
GTK_MODULES=gail:atk-bridge
CLUTTER_IM_MODULE=ibus
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:mi=
XDG_SESSION_DESKTOP=ubuntu
SHLVL=1
LESSCLOSE=/usr/bin/lesspipe %s %s
QT_IM_MODULE=ibus
TERM=xterm-256color
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
```

과제 1 – 매개인자 입력 후 인자에 해당하는 값 출력하기

```
/home/raynor/CLionProjects/ch8-10/cmake-build-debug/ch8_10 -e PATH
-e PATH
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin

Process finished with exit code 0
```

매개인자 –e PATH

과제 1 – 매개인자 입력 후 인자에 해당하는 값 출력하기

매개인자 –u

```
/home/raynor/CLionProjects/ch8-10/cmake-build-debug/ch8_10 -u
-u EMPTY
My Realistic User ID: 1000(raynor)
My Valid User ID: 1000(raynor)

Process finished with exit code 0
```

매개인자 –g

```
/home/raynor/CLionProjects/ch8-10/cmake-build-debug/ch8_10 -g
-g EMPTY
My Realistic Group ID: 1000(raynor)
My Valid Group ID: 1000(raynor)

Process finished with exit code 0
```

과제 1 – 매개인자 입력 후 인자에 해당하는 값 출력하기

매개인자 –i

```
/home/raynor/CLionProjects/ch8-10/cmake-build-debug/ch8_10 -i
-i EMPTY
my process number : [34419]
Process finished with exit code 0
```

매개인자 –p

```
/home/raynor/CLionProjects/ch8-10/cmake-build-debug/ch8_10 -p
-p EMPTY
my parent's process number : [33574]
Process finished with exit code 0
```

과제 2 – 현재 프로세스와 부모프로세스의 관계 파악하기

- 다음 프로그램을 실행시켜 프로그램의 첫 번째 두 번째 출력과 실제 실행 중인 프로세스들을 비교하여 그 관계를 설명하시오. System() 함수는 프로그램 내에서 명령어를 실행시키는 함수이다.

```c
#include <stdio.h>
#include <stdlib.h> //system
#include <unistd.h> //getuid
int main() {
    int pid;

    printf( format: "my process number : [%d] \n", getpid());

    printf( format: "my parent's process number : [%d] \n", getppid());

    system( command: "ps");
    return 0;
}
```

```
/home/raynor/CLionProjects/ch8-12/cmake-build-debug/ch8_12
my process number : [34543]
my parent's process number : [33574]
    PID TTY          TIME CMD
   4300 pts/0    00:00:00 bash
  33559 pts/0    00:00:00 sh
  33574 pts/0    00:03:02 java
  33627 pts/0    00:00:00 fsnotifier
  33636 pts/0    00:00:00 ion.clangd.main
  34251 pts/0    00:00:00 ion.clangd.main
  34543 pts/0    00:00:00 ch8_12
  34544 pts/0    00:00:00 sh
  34545 pts/0    00:00:00 ps
```

과제 3 – 예제 출력을 통해 fork() 시스템 호출의 수행역할 파악하기

- 다음 프로그램의 출력은 무엇인가? 이 출력으로 부터 판단하면 fork() 시스템 호출은 어떤 일을 하였는가?

```c
#include <stdio.h>
#include <unistd.h> //getuid

int main() {

    int pid;
    printf( format: "[1] =%d\n", getpid());
    if((pid = fork()) == 0)
        printf( format: "[2] = %d, %d \n", getpid(), getppid());
    sleep( seconds: 1);

    return 0;
}
```

```
/home/raynor/CLionProjects/ch8-13/cmake-build-debug/ch8_13
[1] =34618
[2] = 34619, 34618
```

E N D





School of Computer Software at Daegu Catholic University © 2021